

Procesamiento de Lenguajes (PL)

Curso 2016/2017

Práctica 5: traductor a código m2r

Fecha y método de entrega

La práctica debe realizarse de forma **individual**, y debe entregarse a través del servidor de prácticas del DLSI **antes de las 23:59 del 10 de junio de 2026**. Los ficheros fuente (“plp5.1”, “plp5.y”, “comun.h”, “TablaSimbolos.h”, “TablaSimbolos.cc”, “TablaTipos.h”, “TablaTipos.cc”, “Makefile” y aquellos que se añadan) se comprimirán en un fichero llamado “plp5.tgz”, sin directorios.

Al servidor de prácticas del DLSI se puede acceder desde la web del DLSI (<http://www.dlsi.ua.es>), en el apartado “Entrega de prácticas”. Una vez en el servidor, se debe elegir la asignatura PL y seguir las instrucciones.

Descripción de la práctica

- La práctica 5 consiste en realizar un compilador para el lenguaje fuente que se describe más adelante (un subconjunto de Java), que genere código para el lenguaje objeto **m2r**, utilizando como en la práctica anterior **bison** y **flex**. Los fuentes deben llamarse **plp5.1** y **plp5.y**. Para compilar la práctica se utilizarán las siguientes órdenes:

```
$ flex plp5.1
$ bison -d plp5.y
$ g++ -o plp5 plp5.tab.c lex.yy.c
```

- El compilador debe diseñarse teniendo en cuenta las siguientes restricciones:
 - En ningún caso se permite que el código objeto se vaya imprimiendo por la salida estándar conforme se realiza el análisis, debe imprimirse al final.
 - Tampoco se permite utilizar ningún *buffer* o *array* global para almacenar el código objeto generado, el código objeto que se vaya generando debe circular por los atributos de las variables de la gramática (en memoria dinámica), y al final, en la regla correspondiente al símbolo inicial, se imprimirá por la salida estándar el código objeto; éste debe ser el único momento en que se imprima algo por la salida estándar. Se recomienda utilizar el tipo **string** para almacenar el código generado.
 - Se aconseja utilizar muy pocas variables globales, especialmente en las acciones semánticas de las reglas; en ese caso siempre es aconsejable utilizar atributos. El uso de variables globales está, en general, reñido con la existencia de estructuras recursivas tales como las expresiones o los bloques de instrucciones. Se pueden utilizar variables globales temporales mientras se usen sea únicamente dentro de una misma acción semántica, sin efectos fuera de ella.
- Si se produce algún tipo de error, léxico, sintáctico o semántico, el programa enviará un mensaje de error de una sola línea a la salida de error (**stderr**). Se proporcionará una función “**msgError**” que se encargará de generar los mensajes de error correctos, a la que se debe pasar la fila, la columna y el lexema del token más relacionado con el error; en el caso de los errores léxicos y sintácticos, el lexema será la cadena “**yytext**” que proporciona el analizador léxico.

Especificación sintáctica

La sintaxis del lenguaje fuente puede ser representada por la siguiente gramática:

<i>S</i>	→	<i>Import Class</i>
<i>Import</i>	→	<i>Import</i> import <i>SecImp</i> pyc
<i>Import</i>	→	ϵ
<i>SecImp</i>	→	<i>SecImp</i> punto id
<i>SecImp</i>	→	<i>SecImp</i> punto scanner
<i>SecImp</i>	→	id
<i>Class</i>	→	public class id llavei <i>Main</i> llaved
<i>Main</i>	→	public static void main pari string cori cord id pard <i>Bloque</i>
<i>Tipo</i>	→	int
<i>Tipo</i>	→	double
<i>Tipo</i>	→	boolean
<i>Bloque</i>	→	llavei <i>BDecl</i> <i>SeqInstr</i> llaved
<i>BDecl</i>	→	<i>BDecl</i> <i>DVar</i>
<i>BDecl</i>	→	ϵ
<i>DVar</i>	→	<i>Tipo</i> <i>LIdent</i> pyc
<i>DVar</i>	→	<i>Tipo</i> <i>DimSN</i> id asig new <i>Tipo</i> <i>Dimensiones</i> pyc
<i>DVar</i>	→	scanner id asig new scanner pari system punto in pard pyc
<i>DimSN</i>	→	<i>DimSN</i> cori cord
<i>DimSN</i>	→	cori cord
<i>Dimensiones</i>	→	cori nentero cord <i>Dimensiones</i>
<i>Dimensiones</i>	→	cori nentero cord
<i>LIdent</i>	→	<i>LIdent</i> coma <i>Variable</i>
<i>LIdent</i>	→	<i>Variable</i>
<i>Variable</i>	→	id
<i>SeqInstr</i>	→	<i>SeqInstr</i> <i>Instr</i>
<i>SeqInstr</i>	→	ϵ
<i>Instr</i>	→	pyc
<i>Instr</i>	→	<i>Bloque</i>
<i>Instr</i>	→	<i>Ref</i> asig <i>Expr</i> pyc
<i>Instr</i>	→	system punto out punto println pari <i>Expr</i> pard pyc
<i>Instr</i>	→	system punto out punto print pari <i>Expr</i> pard pyc
<i>Instr</i>	→	if pari <i>Expr</i> pard <i>Instr</i>
<i>Instr</i>	→	if pari <i>Expr</i> pard <i>Instr</i> else <i>Instr</i>
<i>Instr</i>	→	while pari <i>Expr</i> pard <i>Instr</i>
<i>Expr</i>	→	<i>Expr</i> or <i>EConj</i>
<i>Expr</i>	→	<i>EConj</i>
<i>EConj</i>	→	<i>EConj</i> and <i>ERel</i>
<i>EConj</i>	→	<i>ERel</i>
<i>ERel</i>	→	<i>Esimple</i> relop <i>Esimple</i>
<i>ERel</i>	→	<i>Esimple</i>
<i>Esimple</i>	→	<i>Esimple</i> addop <i>Term</i>
<i>Esimple</i>	→	<i>Term</i>
<i>Term</i>	→	<i>Term</i> mulop <i>Factor</i>
<i>Term</i>	→	<i>Factor</i>
<i>Factor</i>	→	<i>Ref</i>
<i>Factor</i>	→	id punto nextint pari pard
<i>Factor</i>	→	id punto nextdouble pari pard
<i>Factor</i>	→	nentero
<i>Factor</i>	→	nreal
<i>Factor</i>	→	ctebool
<i>Factor</i>	→	pari <i>Expr</i> pard
<i>Factor</i>	→	not <i>Factor</i>
<i>Factor</i>	→	pari <i>Tipo</i> pard <i>Factor</i>
<i>Ref</i>	→	id
<i>Ref</i>	→	<i>Ref</i> cori <i>Esimple</i> cord

Especificación léxica

EXPRESIÓN REGULAR	COMPONENTE LÉXICO	VALOR LÉXICO ENTREGADO
[\n\t]+	(ninguno)	
boolean	boolean	(palabra reservada)
int	int	(palabra reservada)
double	double	(palabra reservada)
main	main	(palabra reservada)
System	system	(palabra reservada)
out	out	(palabra reservada)
in	in	(palabra reservada)
print	print	(palabra reservada)
println	println	(palabra reservada)
String	string	(palabra reservada)
class	class	(palabra reservada)
import	import	(palabra reservada)
new	new	(palabra reservada)
public	public	(palabra reservada)
static	static	(palabra reservada)
void	void	(palabra reservada)
Scanner	scanner	(palabra reservada)
nextInt	nextInt	(palabra reservada)
nextDouble	nextdouble	(palabra reservada)
if	if	(palabra reservada)
else	else	(palabra reservada)
while	while	(palabra reservada)
true	ctebbool	(palabra reservada)
false	ctebbool	(palabra reservada)
[A-Za-z][0-9A-Za-z]*	id	(nombre del ident.)
[0-9]+	nentero	(valor numérico)
([0-9]+) "." ([0-9]+)	nreal	(valor numérico)
,	coma	
;	pyc	
.	punto	
(	pari	
)	pard	
==	relop	==
!=	relop	!=
<	relop	<
<=	relop	<=
>	relop	>
>=	relop	>=
+	addop	+
-	addop	-
*	mulop	*
/	mulop	/
=	asig	
[	cori	
]	cord	
{	llavei	
}	llaved	
&&	and	
	or	
!	not	

Notas:

1. El analizador léxico también debe ignorar los blancos¹ y los comentarios, que comenzarán por ‘//’ y terminarán al final de la línea.
2. Hay componentes léxicos y reglas de la gramática cuyo único objetivo es conseguir que un programa en este lenguaje fuente pueda ser compilado también con un compilador de Java, como por ejemplo la importación de paquetes. En muchos casos, la práctica producirá un error sintáctico cuando un compilador de Java produciría un error semántico.

¹Los tabuladores se contarán como un espacio en blanco.

Especificación semántica

Las reglas semánticas de este lenguaje son similares a las de Java, en muchos casos es necesario hacer conversiones de tipos, y en algunos casos se debe producir un mensaje de error semántico cuando aparezca alguna construcción no permitida. Las reglas se pueden resumir como:

Declaración de variables

- No es posible declarar dos veces un símbolo en el mismo ámbito con el mismo nombre, aunque sea con el mismo tipo (recuérdese que no se debe distinguir entre mayúsculas y minúsculas). Además, en Java no está permitido declarar en un ámbito una variable con el mismo nombre que otra variable de un ámbito exterior (no se permite *ocultar* variables).
- No es posible utilizar una variable sin haberla declarado previamente. Se pueden declarar variables de tipo simple (entero, real o booleano), variables de tipo **Scanner**, y arrays.
- Las variables de tipo **Scanner** sólo pueden utilizarse para leer con los métodos **nextInt** y **nextDouble**, nada más. Además, dichos métodos sólo pueden utilizarse con variables de tipo **Scanner**, como es lógico.
- Si al declarar una variable el espacio ocupado por ésta sobrepasa el tamaño máximo de memoria para variables (siempre inferior a 16384, que es el tamaño de la memoria de la máquina virtual), el compilador debe producir un mensaje de error indicando el lexema exacto de la variable que ya no cabe en memoria. El tamaño de todos los tipos básicos es 1, es decir, una variable simple ocupa una única posición, ya sea de tipo entero, real o booleano.
- De las 16384 posiciones de memoria de la máquina virtual, se deben destinar 16000 a variables declaradas por el programa fuente, y 384 a variables temporales.

Instrucciones

Asignación: en esta instrucción tanto la referencia que aparece a la izquierda del operador “=” como la expresión de la derecha deben ser del mismo tipo. Solamente hay una excepción a esta regla, y consiste en que está permitida también la asignación cuando la referencia es real y la expresión es entera (que debe convertirse a real antes de ser asignada). No es posible asignar valor a las variables de tipo **Scanner** (excepto en la declaración, por supuesto).

Lectura y escritura: al escribir valores booleanos se escribirá un 0 si es **false**, y un 1 si es **true**. Solamente es posible leer valores enteros o reales, y para ello se llamará a los métodos **nextInt** y **nextDouble** utilizando una variable de tipo **Scanner**.

Control de flujo: las instrucciones “**if**” y “**while**” tienen una semántica similar a la de Java, se exige que el tipo de la expresión sea booleano; en caso contrario, se debe producir un error semántico. Es posible que tengas que modificar la gramática para dar el error lo antes posible en esas reglas.

Bloques: las declaraciones de variables sólo son válidas en el cuerpo del bloque, es decir, una vez que se cierra el bloque, el compilador debe *olvidar* todas las variables declaradas en él. Además, no es posible declarar en un bloque variables con el mismo nombre que las variables de otros bloques de niveles superiores (en C/C++ sí está permitido, en Java no).

Expresiones

Operadores booleanos: en los operadores “**||**”, “**&&**” y “**!**” los operandos deben ser booleanos. Por simplificar, la evaluación de los operadores “**||**” y “**&&**” debe ser como el resto de operadores binarios, es decir, no debe implementarse la evaluación en cortocircuito presente en Java y otros lenguajes derivados de C.

Operadores relacionales: Estos operadores permiten comparar valores numéricos (enteros o reales) entre sí, y también valores booleanos entre sí (en cuyo caso se considera que “**false**” es menor que “**true**”). No se permite comparar valores de distinto tipo (excepto valores enteros y reales, por supuesto). El resultado de una operación relacional siempre es de tipo booleano.

Operadores aritméticos: solamente pueden utilizarse con valores enteros o reales. La división entre valores enteros siempre es una división entera; si alguno de los operandos es real, la división es real (y puede ser necesario convertir uno de los operandos a real).

Arrays:

- En las declaraciones de variables de tipo *array*, el número de dimensiones del tipo debe coincidir con el número de dimensiones del array creado con `new`, y los tipos deben ser iguales; además, los números entre corchetes deben ser estrictamente mayor que cero (aunque Java permite arrays de tamaño 0). El rango de posiciones del *array* será, como en Java, de 0 a $n - 1$.
- No se permite utilizar una variable de tipo *array* con más ni con menos índices de los necesarios (según la declaración de la variable) para obtener un valor de tipo simple.
- No se permite poner índices (corchetes) a variables que no sean de tipo *array*.
- No está permitida la asignación entre valores de tipo *array*. Las asignaciones deben realizarse siempre con valores de tipo simple.
- La expresión entre corchetes (el índice) debe ser de tipo entero.
- En principio, el número de dimensiones que puede tener una variable de tipo *array* no está limitado más que por la memoria disponible. Por este motivo es aconsejable utilizar una tabla de tipos.

Conversión de tipo: es posible, utilizando un *cast*, convertir un factor de una expresión de un tipo simple (entero, real o booleano) a otro; en ese caso, los valores `true` y `false` se convertirían a 1 y 0, respectivamente, y los valores numéricos se convertirán a `true` si son distintos de 0, y a `false` si son 0.

Mensajes de error

En la web de la asignatura se publicará un fichero con el código de una función para emitir mensajes de error (léxico, sintáctico y semántico). Los errores semánticos son:

ERR_YADECL: se emite cuando ya se ha declarado una variable con el mismo identificador (en el ámbito actual o en un ámbito exterior). El lexema, la fila y la columna se deben referir al identificador que se va a declarar.

ERR_NODECL: se emite cuando se utiliza un identificador y no ha sido declarado.

ERR_NOSC: este error se produce cuando se intenta utilizar los métodos `nextInt` o `nextDouble` con una variable que no es de tipo `Scanner`

ERR_SCVAR: este error se produce cuando se intenta utilizar una variable de tipo `Scanner` (sin la llamada a los métodos `nextInt` o `nextDouble`) en una expresión o en una asignación.

ERR_TIPOSDECLARRAY: se emite cuando, en una declaración de array, los tipos antes y después de `new` no coinciden. El lexema, la fila y la columna se deben referir al identificador que se va a declarar.

ERR_DIMSDECLARRAY: se emite cuando, en una declaración de array, el número de dimensiones antes y después de `new` no coinciden.

ERR_DIM: se emite cuando, en una declaración de array, alguna de las dimensiones no es mayor que 0. El lexema, la fila y la columna se deben referir al número.

ERR_TIPOSASIG: si en una asignación los tipos de la referencia y la expresión no son compatibles, se emite este error. El lexema, la fila y la columna se deben referir al operador de asignación.

ERR_TIPOS: se emite cuando los dos operandos de un operador relacional no son compatibles. El lexema, la fila y la columna se deben referir al operador relacional.

ERR_TIPOSIFW: se emite cuando la expresión de un `if` o un `while` no es de tipo booleano. El lexema, la fila y la columna se deben referir al `if` o al `while`.

ERR_OPNOBOOL: se produce cuando alguno de los operandos de un operador booleano no es de tipo booleano. El lexema, la fila y la columna se deben referir al operador booleano.

ERR_NUM: se produce cuando alguno de los operandos de un operador numérico no es de tipo entero o real. El lexema, la fila y la columna se deben referir al operador.

ERR_FALTAN: se produce cuando en una referencia a un array faltan índices por poner. El lexema, la fila y la columna se deben referir al último “]”, o bien al identificador si no hay corchetes.

ERR_SOBRAN: se produce cuando en una referencia a un array sobran índices. El lexema, la fila y la columna se deben referir al primer “[” que sobra.

ERR_EXP_ENT: se produce cuando en una referencia a un array la expresión entre corchetes no es de tipo entero.

ERR_NOCABE: este error se debe emitir cuando se declara una variable que no cabe en el espacio de memoria de la máquina virtual (16000 posiciones).

ERR_MAXTMP: este error se debe emitir cuando no es posible obtener ninguna variable temporal más, porque se han agotado las 384 posiciones disponibles. El lexema, la fila y la columna pueden tener cualquier valor.